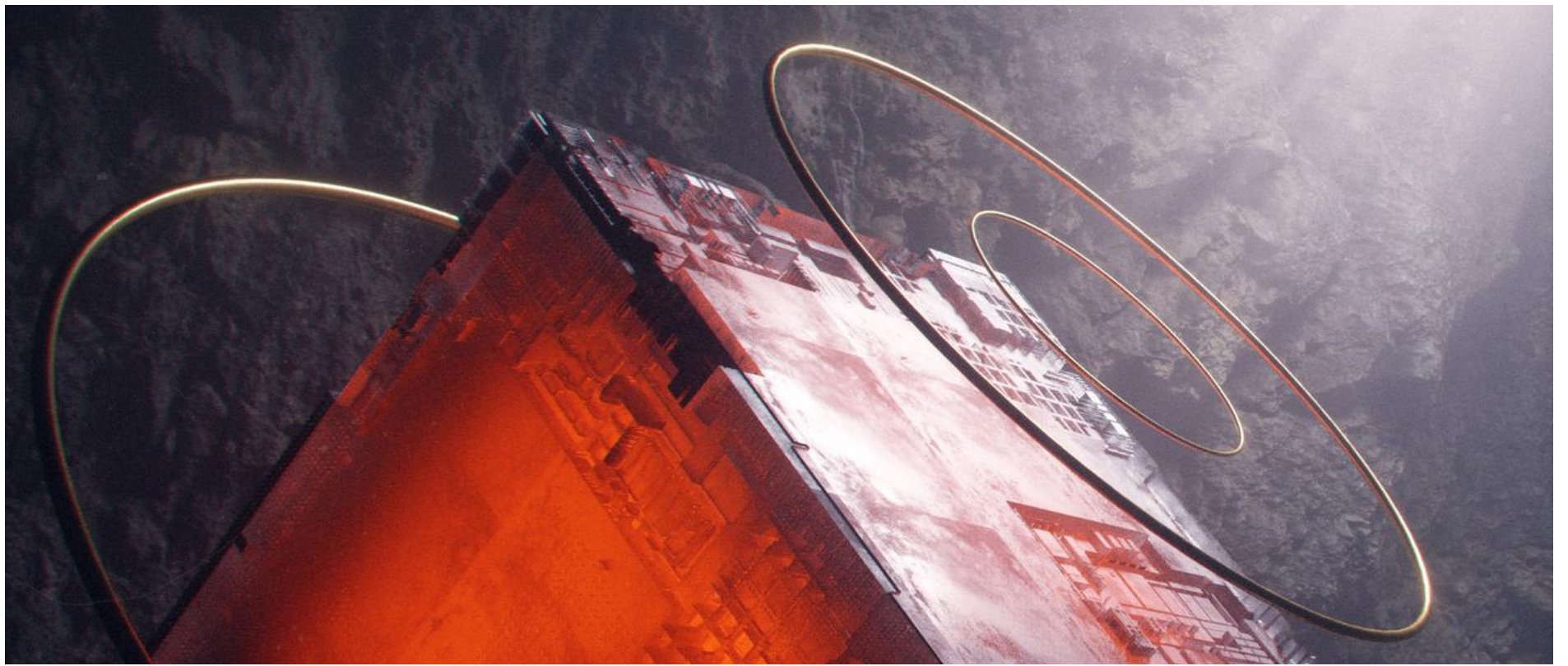




金三银四面试突击ALL IN ONE 第四篇 —— 25年金三银四面试高频踩坑解析

1. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 -- Typescript: JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能 Vue 应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂 React 最佳实践，手把手从 0 打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线 25K 前端 P6 岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节 T2-1 岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市 25K 前端 P6 岗的满分简历](#)
- [📖 金三银四面试突击 ALL IN ONE 第三篇 —— 大厂 P6 级模拟面试来啦](#)

2. 简历

作为面试的敲门砖，大/中/小厂的简历分别有什么异同点？如果在简历面上胜人一筹？

2.1 简历内容详细分析

简历模板的选择很讲究，有些简历基本不看内容就会被刷掉，这些简历一般会对面试官进行视觉攻击，让简历给面试官的第一印象就是反感。

有两种**令人反感的简历模板**：

1. 一种是经典简历模板，真是堪称经典，配色比较魔幻，加上表格类的简历属于 low 到底端的简历类型，基本上扫一眼就扔了，这种简历只需要 3 秒钟就能被面试官扔到垃圾堆。

2. 另一种是设计感十足的简历模板，这种简历设计感十足，这五颜六色的配色能亮瞎面试官的双眼，这种花里胡哨的简历同样也是3秒钟沉到垃圾堆底部的简历。

以上两类简历模板堪称面试官杀手，只要用了上述两类模板，绝对连让面试官看第二眼的兴趣都没有。面试官筛简历要的是**高效、清晰、内容突出**，不管是HR还是技术面试官都想在最快速的情况下看到有效信息，你眼中所谓的『视觉效果』在别人眼里就是『视觉噪音』或者『视觉垃圾』，严重影响看简历的心情和寻找有效信息的速度。

2.1.1 准备个人信息

个人信息部分主要包括**姓名、电话、电子邮箱、求职意向**，当然这四个是必填的，其它的都是选填，填好了是加分项，否则很可能减分。

接下来才是重点：

1.**博客**：如果准备一个基本没有更新的博客或者没有任何贡献的github，那么给面试官一种为了放上去而放上去的感觉，这基本上就是在跟面试官说『**这个候选人平时根本没有总结提炼的习惯**』，所以如果有长期维护的github或者博客一定要放上去，质量好的话会非常有用，如果没有千万别放。

2.**学历**：如果你的学历是专科、高中毕业之类的，还写在简历头部强调一遍，这就增加了简历被刷的几率，如果是研究生或者985211学历可以写，突出一下学历优势，本科学历在技术面试领域基本上敲门砖级别的，没必要写。

3.**年龄**：如果是大龄程序员，尤其是还在求一份低端岗位的时候千万别写，一个大龄程序员在求职一个中低端岗位，说明这些年基本原地踏步，还不能加班，到这里基本上此简历就凉了一半了。

4.**照片**：形象优秀的可以贴，尤其是形象优秀的女程序媛，其它的最好不要贴，如果要贴的话，最好是贴那种PS过的非常职业的证件照。

2.1.2 准备专业技能

对于程序员的专业技能其实就是技术栈，对于自己的技术栈如何描述是个很难的问题，比如什么算是精通？什么算是了解？什么是熟悉？

关于对技术技能的描述有很多种，这里只讲三种的了解、熟悉、精通：

●**了解**：使用过某一项技术，能在别人指导下完成工作，但不能胜任复杂工作，也不能独立解决问题。

●**熟悉**：大量运用过的某一项技术，能独立完成工作，且能独立完成有一定复杂度的工作，在技术的应用层面不会有太大问题，甚至理解一点原理。

●**精通**：不仅可以运用某一门技术完成复杂项目，而且理解这项技术背后的原理，可以对此技术进行二次开发，甚至本身就是技术源码的贡献者。

以Vue框架为例，如果可以用Vue写简单的页面，单独完成某几个页面的开发，但是无法脱离公司脚手架工作，也无法独立从0完成一个有一定复杂度的项目，只能称之为了解。

如果有大量运用vue的经验，有从0独立完成一定复杂度项目的能力，可以完全脱离脚手架进行开发，且对vue的原理有一定的了解，可以称之为熟悉。

如果用vue完成过复杂度很高的项目，而且非常熟悉vue的原理，是vue源码的主要贡献者，亦或者根据vue源码进行过魔改（比如mpvue），可以称得上精通。

那么有两个坑是候选人经常犯的，**杂和精**，这种两个坑大量集中在应届生和刚毕业没两年的新手身上，其主要特点是急于表现自我、对技术深度与广度出现无知而导致的过度自信。

首先说杂，比如要应聘一个Vue前端，老老实实把自己的Vue技术栈写好就行了，强调一下自己擅长什么即可，最好专精某领域比如后台管理系统实践等。这时一些简历非要给自己加戏，自己会的不会的一股脑往上堆，什么工程化定制、各个系统从0到1落地、AI都要体现出来，越杂越好，这种简历给人的印象就是个什么都不懂的半吊子。

再说精，一个刚毕业的应届生，出来简历就各种精通，精通Vue、精通Vue源码、精通React源码等等，这种简历是不会没头没脑投过来的，这种在大学里就精通各种的天才早被他的各种学长介绍进了大厂或者外企做某某Star重点培养了，往往看到的这种也是半吊子。

2.1.3 准备工作经历

工作经历本身不用花太多笔墨去写，面试官主要想看的就是每段工作经历的持续时间、在不同公司担任的职责如何、是否有大厂的工作经验等。

那么什么简历在这里给面试官减分呢？

●**频繁跳槽**：比如三年换了三家公司，每个公司呆的时长不要超过一年；

●**常年初级岗**：比如工作五六年之后依然在完成一些简单的项目开发；

●**末流公司经历**：在技术招聘届，大厂的优先级最高比如BAT、TMD甚至微软、谷歌等外企，知名度独角兽其次，一般的互联网公司排在第三，就是工作中小型的互联网公司一般大家叫不上名字，排在最后的就是外包和传统企业的经历；

(1) 所以，如果你有频繁跳槽的经历怎么办？在本公司老老实实等到满一年再跳槽。

(2) 如果常年初级岗怎么办？想办法晋升或者参与一些业界知名项目，又或者写一个有一定复杂度的私人项目。

(3) 如果有末流公司经历怎么办？如果是很久以前的末流公司经验可以直接不写，也没人在乎你很早之前的工作经历，如果你现在就在末流公司，赶紧想办法跳槽，去不了大厂，去非知名的互联网公司也算是胜利大逃亡了。

2.1.4 准备项目经历

项目经历不管对于社招还是校招都是重中之重，很多时候成败就在于项目经历这块，一个普通本科可以通过优秀的项目经历逆袭985，一个小厂的员工也可以获得大厂的面试机会。但是必须要说一下项目经历的编写很讲究，这是为后面面试部分铺路的绝佳机会，也是直接让你的简历扑街的重点沦陷区域。

先来看简历中项目经历的坑。

2.1.4.1 切忌流水账写法

项目经历流水账写法是绝大多数简历的通病，通篇下来就讲了一件事『我干了啥』。

大部分简历却是这样的：

用Vue、Vuex、Vue-router、axios等技术开发电商网站的前端部分，主要负责首页、店铺详情、商品详情、商品列表、订单详情、订单中心等相关页面的开发工作，与设计师与后端配合，可要高度还原设计稿。

这个描述有什么问题？其实看似也没啥问题，但是这种流水账写法太多了，完全无法突出自己的优势展现自己的能力。**项目经历是考察重点，面试官想知道候选人在一次项目经历中扮演的角色、负责的模块、碰到的问题、解决的思路、达成的效果以及最后的总结与沉淀。**

而上面的描述只显示了我干了啥，所以这种项目描述几乎是没意义的，因为对于面试官而言他看不到有效信息，没有有效信息的项目描述基本就没价值了，如果这个时候你还没有大厂经历或者名校背书，基本上也就凉了。

2.1.4.2 切忌堆积项目

堆积项目这种现象往往出现在没有什么优秀项目经历的简历身上，候选人企图以数量优势掩盖质量的劣势，其实往往适得其反，项目经历的一栏最好放2-3个项目，非常优秀的项目可能放一个就足够了，举个极端例子如果有一天尤雨溪写简历，其实只需要在项目经历那些一行Vue.js作者就行了,当然,他并不需要投简历。

有一些项目切忌放上去：

●**demo级项目**：很多简历居然还在放一些仿xx官网的demo，这是十足的减分项，有一些则是东拼西凑抄了一些框架的源码搞了个玩具项目，没有任何价值。

●**烂大街的项目**：这种以vue技术栈的为最，由于视频网站的某门课程流行，导致大量的仿饿了么、仿qq音乐、仿美团、仿去哪儿，十份简历5份一模一样的项目，你是面试官会怎么想。

●**低质量的开源项目**：一个大原则就是低star的尽量别放（除非是高质量代码的冷门项目），长期弃坑的也不要放，不要为了凑数量把低质量的项目暴露出来，好好藏着。

如果只放两个项目，最好的搭配是一个公司内部挑大梁的项目和一个社区内的开源项目，后者之所以可以占据一席之地，是因为通过你的开源项目，面试官可以通过commit完整看到你的创造过程，比如工程化建设、commit规范、代码规范、协作方式、代码能力、沟通能力等等，这甚至比面试都有用，没有比开源项目更能展示你综合素质的东西了。

2.1.4.3 切忌放虚假项目

一个项目做没做过只要是有经验的面试官一问便知，如果你真的靠假项目忽悠过了面试，那这个公司八成也有问题，人才把关不过硬，你可以想象你的队友都是什么水平，在这种公司大成长价值也不大。好，如果你说实在没项目可写了，我只能造假了，那么你应该想一下这多层追问。

比如你说你优化了一个前端项目的首屏性能，降低了白屏时间，那么面试官对这个性能优化问题会进行深挖，来考察候选人的实际水平：

- 你的性能优化指标是怎么确定的？平均下来时间减短了多少？
- 你的性能是如何测试的？有两种主流的性能测试方法你是怎么选的？
- 你是根据哪些指标进行针对性优化的？
- 除了你说的这些优化方法还有没有想过通过xx来解决？
- 你的这个优化方法在实际操作中碰到过什么问题吗？有没有进一步做过测试？
- 我们假设这么一种情况，比如xxxx，你会这么进行优化？

面试官多层追问的逻辑是这样的：**了解背景 -> 了解方案 -> 深挖方案 -> 模拟场景**

首先得了解你性能优化的指标如何，接着需要了解你是这么测试的指标、再怎么进行针对性优化的，再接着提出一些其它解决方案考察你对优化场景的知识储备和方案决策能力，最后再模拟一个其它的业务场景，来考察你的技能迁移能力，看看是否是对某块领域有一定的了解，而不是只针对某个项目。

如果要真的在面试现场对答如流，那么一定是在某一块领域有一定知识储备的人，不是随随便便搞个项目就能蒙混过关的。

2.1.4.4 合格的项目经历

合格的项目经历必须要有以下几点：

- 项目概述
- 个人职责
- 项目难点
- 工作成果

如果不怕字太多，还可以选择性加入解决方案、选型思路等，但是由于篇幅限制和为面试铺垫就不太建议写得太多。

项目概述的目的是让面试官理解项目，不是每个人面试官都做过你的那种项目，所以需要个简述方便面试官理解。

个人职责就是告诉面试官你在本项目中扮演的角色，是领导者？主导者？还是跟随者，你负责了哪些模块，承担了多大的工作量，以此来评估你在团队中的作用。

项目难点的目的在于让面试官看到你碰到的技术难题，方便后续面试对项目进行一系列讨论。

工作成果就很明显了，面试官需要看到你在做了上述工作到底达成了什么成绩，这个时候最好以数据说话，比如访问量、白屏时间等等。

这个时候也切忌长篇大论，把技术细节都写上去，甚至还写了心路历程的都是大忌，一方面篇幅太大会造成视觉混乱，另一方面面试官想看到的是简历，不是技术总结，面试官要面对上百份简历没那么时间来看你长篇大论，长篇大论大可以在面试中展开。

最好的方法就是一行文字简单得说清楚即可，反正项目面的时候会问到，到时候把准备的内容讲给面试官，掌握面试的主动权就是从项目经历这一栏中开始。

2.1.4.5 其他

- **教育背景**

应届生可以写得更详细一点，比如绩点排名怎么样，有没有突出的科目，社招就不要写太多了，简单的入学时间、学校、专业即可，而且写最高学历即可，没必要从初中就开始写学历流水账，没有人看的。

- **注意事项**

自我评价不建议写：技术面试几乎没人看你的自我评价，连面试技术问题都嫌『talk is cheap show me the code』，你的自我评价除了占篇幅没啥用处，充其量算是面试官的干扰信息。

- **简历封面千万别搞**

这都是一些简历制作网站骗用户付费的伎俩，不仅是互联网行业，其它行业我也没见过要简历封面这种无用操作的。

- **证书不建议写**

应届生可以酌情考虑弄个六级证书什么的，对于社招而言，列一堆证书甚至是减分项，国内的各种证你也懂的，是有多不自信才沦落到靠一堆证书来证明自己的价值。

- **千万别用技能图表**

首先用90分、80分来评价自己的技术本身就没有什么说服力，也不可能这么精准，而且什么是90分、什么是80根本就没有一个公论，所以用一般的比较通用的熟悉、精通描述即可，千万别加戏，面试官或者HR没那么多闲工夫去理解你的图表，老老实实按最通用高效的方式描述自己的技术栈。

- **简历最好不超过2页**

程序员又不是设计师有时候需要作品呈现，如果简历超过一页一定是出问题了，项目、技术栈描述太多太杂占据了大量篇幅。

2.2 真实简历分析&优化

飞书链接：[📖 13. 如何写出高质量的简历（For 付费学员）](#) 密码：86t&9493

3. 具体技术栈应该掌握到什么程度？



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7



4. 前端高频算法考点

4.1 双指针

4.1.1 接近的三数之和

给定一个包括 n 个整数的数组 `nums` 和一个目标值 `target`。找出 `nums` 中的三个整数，使得它们的和与 `target` 最接近。返回这三个数的和。假定每组输入只存在唯一答案。

Code block

```
1  示例：
2
3  输入：nums = [-1,2,1,-4]，target = 1
4  输出：2
5  解释：与 target 最接近的和是 2 (-1 + 2 + 1 = 2) 。
```

提示：

`3 <= nums.length <= 10^3`

`-10^3 <= nums[i] <= 10^3`

`-10^4 <= target <= 10^4`

链接：<https://leetcode-cn.com/problems/3sum-closest>

先按照升序排序，然后分别从左往右依次选择一个基础点 `i` (`0 <= i <= nums.length - 3`)，在基础点的右侧用双指针去不断的找最小的差值。

假设基础点是 `i`，初始化的时候，双指针分别是：

- `left`：`i + 1`，基础点右边一位。
- `right`：`nums.length - 1` 数组最后一位。

然后求此时的和，如果和大于 `target`，那么可以把右指针左移一位，去试试更小一点的值，反之则把左指针右移。

在这个过程中，不断更新全局的最小差值 `min`，和此时记录下来的和 `res`。

最后返回 `res` 即可。

Code block

```
1  /**
2   * @param {number[]} nums
3   * @param {number} target
4   * @return {number}
5   */
6  let threeSumClosest = function (nums, target) {
7      let n = nums.length
8      if (n === 3) {
9          return getSum(nums)
10     }
11     // 先升序排序 此为解题的前置条件
12     nums.sort((a, b) => a - b)
13
14     let min = Infinity // 和 target 的最小差
15     let res
16
17     // 从左往右依次尝试定一个基础指针 右边至少再保留两位 否则无法凑成3个
18     for (let i = 0; i <= nums.length - 3; i++) {
19         let basic = nums[i]
20         let left = i + 1; // 左指针先从 i 右侧的第一位开始尝试
21         let right = n - 1 // 右指针先从数组最后一项开始尝试
```

```

22
23     while (left < right) {
24         let sum = basic + nums[left] + nums[right] // 三数求和
25         // 更新最小差
26         let diff = Math.abs(sum - target)
27         if (diff < min) {
28             min = diff
29             res = sum
30         }
31         if (sum < target) {
32             // 求出的和如果小于目标值的话 可以尝试把左指针右移 扩大值
33             left++
34         } else if (sum > target) {
35             // 反之则右指针左移
36             right--
37         } else {
38             // 相等的话 差就为0 一定是答案
39             return sum
40         }
41     }
42 }
43
44 return res
45 };
46
47 function getSum(nums) {
48     return nums.reduce((total, cur) => total + cur, 0)
49 }

```

4.1.2 通过删除字母匹配到字典里最长单词

给定一个字符串和一个字符串字典，找到字典里面最长的字符串，该字符串可以通过删除给定字符串的某些字符来得到。如果答案不止一个，返回长度最长且字典顺序最小的字符串。如果答案不存在，则返回空字符串。

Code block

```

1  示例 1:
2
3  输入:
4  s = "abpcplea", d = ["ale","apple","monkey","plea"]
5
6  输出:
7  "apple"
8  示例 2:
9
10 输入:
11 s = "abpcplea", d = ["a","b","c"]
12
13 输出:
14 "a"
15 说明:
16
17 所有输入的字符串只包含小写字母。
18 字典的大小不会超过 1000。
19 所有输入的字符串长度不会超过 1000。

```

链接: <https://leetcode-cn.com/problems/longest-word-in-dictionary-through-deleting>

为数组 `d` 中的**每个**单词分别生成**指针**，指向那个单词目前**已经匹配到**的字符下标。然后对 `s` 进行一次遍历，每轮循环中都对 `d` 中所有指针的**下一位**进行匹配，如果匹配到了，则那个单词的指针后移。

一旦某个单词指针移动到那个字母的最后一位了，就和全局的 `find` 变量比较，先比较长度，后比较字典序，记录下来，最后返回即可。

Code block

```
1  /**
2   * @param {string} s
3   * @param {string[]} d
4   * @return {string}
5   */
6  let findLongestWord = function (s, d) {
7      let n = d.length
8      let points = Array(n).fill(-1)
9
10     let find = ""
11     for (let i = 0; i < s.length; i++) {
12         let char = s[i]
13         for (let j = 0; j < n; j++) {
14             let targetChar = d[j][points[j] + 1]
15             if (char === targetChar) {
16                 points[j]++
17                 let word = d[j]
18                 let wl = d[j].length
19                 if (points[j] === wl - 1) {
20                     let fl = find.length
21                     if (wl > fl || (wl === fl && word < find)) {
22                         find = word
23                     }
24                 }
25             }
26         }
27     }
28
29     return find
30 }
```

4.1.3 搜索二维矩阵

编写一个高效的算法来搜索 $m \times n$ 矩阵 `matrix` 中的一个目标值 `target`。该矩阵具有以下特性：

每行的元素从左到右升序排列。

每列的元素从上到下升序排列。

示例：

现有矩阵 `matrix` 如下：

Code block

```
1  [
2    [1, 4, 7, 11, 15],
3    [2, 5, 8, 12, 19],
4    [3, 6, 9, 16, 22],
5    [10, 13, 14, 17, 24],
6    [18, 21, 23, 26, 30],
7  ]
```

给定 `target = 5`，返回 `true`。

给定 `target = 20`，返回 `false`。

链接：<https://leetcode-cn.com/problems/search-a-2d-matrix-ii>

由题可得，当某一个单元格的值大于目标值的时候，那么目标值一定在这个单元格**上方**的行里。

如果单元格的值小于目标值的时候，那么目标值一定在这个单元格**右边**的列里。

根据这一特性，从最左下角那一格开始，设立**行指针** `row = y - 1`，设立**列指针** `column = 0`，然后不断根据上面描述特征移动这两个指针，直到找到目标值位置。如果超出边界还没有找到目标值，那么就返回 `false`。

Code block

```
1  /**
2   * @param {number[][]} matrix
3   * @param {number} target
4   * @return {boolean}
5   */
6  let searchMatrix = function (matrix, target) {
7      let y = matrix.length
8      if (!y) return false
9      let x = matrix[0].length
10
11     let row = y - 1
12     let column = 0
13     while (row >= 0 && column < x) {
14         let val = matrix[row][column]
15         if (val > target) {
16             row--
17         } else if (val < target) {
18             column++
19         } else if (val === target) {
20             return true
21         }
22     }
23     return false
24 }
```

4.1.4 判断子序列

给定字符串 `s` 和 `t`，判断 `s` 是否为 `t` 的子序列。

你可以认为 `s` 和 `t` 中仅包含英文小写字母。字符串 `t` 可能会很长（长度 $\sim 500,000$ ），而 `s` 是个短字符串（长度 ≤ 100 ）。

字符串的一个子序列是原始字符串删除一些（也可以不删除）字符而不改变剩余字符相对位置形成的新字符串。（例如，"ace"是"abcde"的一个子序列，而"aec"不是）。

Code block

```
1  示例 1:
2  s = "abc", t = "ahbgdc"
3
4  返回 true.
5
6  示例 2:
7  s = "axc", t = "ahbgdc"
8
9  返回 false.
```

后续挑战：

如果有大量输入的 S ，称作 $S_1, S_2, \dots, S_k$ 其中 $k \geq 10$ 亿，你需要依次检查它们是否为 T 的子序列。在这种情况下，你会怎样改变代码？

链接：<https://leetcode-cn.com/problems/is-subsequence>

判断字符串 s 是否是字符串 t 的子序列，我们可以建立一个 i 指针指向「当前 s 已经在 t 中成功匹配的字符下标后一位」。之后开始遍历 t 字符串，每当在 t 中发现 i 指针指向的目标字符时，就可以把 i 往后前进一位。

- 一旦 `i === t.length`，就代表 t 中的字符串全部按顺序在 s 中找到了，返回 `true`。
- 当遍历 s 结束后，就返回 `false`，因为 i 此时并没有成功走 t 的最后一位。

Code block

```
1  /**
2   * @param {string} s
3   * @param {string} t
4   * @return {boolean}
5   */
6  let isSubsequence = function (s, t) {
7      let sl = s.length
8      if (!sl) {
9          return true
10     }
11
12     let i = 0
13     for (let j = 0; j < t.length; j++) {
14         let target = s[i]
15         let cur = t[j]
16         if (cur === target) {
17             i++
18             if (i === sl) {
19                 return true
20             }
21         }
22     }
23
24     return false
25 };
```

4.1.5 分发饼干

假设你是一位很棒的家长，想要给你的孩子们一些小饼干。但是，每个孩子最多只能给一块饼干。对每个孩子 i ，都有一个胃口值 g_i ，这是能让孩子们满足胃口的饼干的最小尺寸；并且每块饼干 j ，都有一个尺寸 s_j 。如果 $s_j \geq g_i$ ，我们可以将这个饼干 j 分配给孩子 i ，这个孩子会得到满足。你的目标是尽可能满足越多数量的孩子，并输出这个最大数值。

注意：

你可以假设胃口值为正。

一个小朋友最多只能拥有一块饼干。

Code block

```
1  示例 1:
2
3  输入: [1,2,3], [1,1]
4
5  输出: 1
6
7  解释:
8  你有三个孩子和两块小饼干，3个孩子的胃口值分别是：1,2,3。
9  虽然你有两块小饼干，由于他们的尺寸都是1，你只能让胃口值是1的孩子满足。
```

```
10  所以你应该输出1。
11  示例 2：
12
13  输入：[1,2]，[1,2,3]
14
15  输出：2
16
17  解释：
18  你有两个孩子和三块小饼干，2个孩子的胃口值分别是1,2。
19  你拥有的饼干数量和尺寸都足以让所有孩子满足。
20  所以你应该输出2。
```

链接：<https://leetcode-cn.com/problems/assign-cookies>

把饼干和孩子的需求都排序好，然后从最小的饼干分配给需求最小的孩子开始，不断的尝试新的饼干和新的孩子，这样能保证每个分给孩子的饼干都恰到好处的不浪费，又满足需求。

利用双指针不断的更新 `i` 孩子的需求下标和 `j` 饼干的值，直到两者有其一达到了终点位置：

1. 如果当前的饼干不满足孩子的胃口，那么把 `j++` 寻找下一个饼干，不用担心这个饼干被浪费，因为这个饼干更不可能满足下一个孩子（胃口更大）。
2. 如果满足，那么 `i++；j++；count++` 记录当前的成功数量，继续寻找下一个孩子和下一个饼干。

Code block

```
1  /**
2   * @param {number[]} g
3   * @param {number[]} s
4   * @return {number}
5   */
6  let findContentChildren = function (g, s) {
7      g.sort((a, b) => a - b)
8      s.sort((a, b) => a - b)
9
10     let i = 0
11     let j = 0
12
13     let count = 0
14     while (j < s.length && i < g.length) {
15         let need = g[i]
16         let cookie = s[j]
17
18         if (cookie >= need) {
19             count++
20             i++
21             j++
22         } else {
23             j++
24         }
25     }
26
27     return count
28 }
```

4.1.6 验证回文串

给定一个字符串，验证它是否是回文串，只考虑字母和数字字符，可以忽略字母的大小写。

说明：本题中，我们将空字符串定义为有效的回文串。

示例 1:

Code block

```
1  输入: "A man, a plan, a canal: Panama"
2  输出: true
```

示例 2:

Code block

```
1  输入: "race a car"
2  输出: false
```

<https://leetcode-cn.com/problems/valid-palindrome>

先根据题目给出的条件，通过正则把不匹配字符去掉，然后转小写。

建立双指针 `i`，`j` 分别指向头和尾，然后两个指针不断的向中间靠近，每前进一步就对比两端的字符串是否相等，如果不相等则直接返回 `false`。

如果直到 `i >= j` 也就是指针对撞了，都没有返回 `false`，那就说明符合「回文」的定义，返回 `true`。

Code block

```
1  /**
2   * @param {string} s
3   * @return {boolean}
4   */
5  let isPalindrome = function(s) {
6      s = s.replace(/[^0-9a-zA-Z]/g, '').toLowerCase()
7      let i = 0
8      let j = s.length - 1
9
10     while(i < j) {
11         let head = s[i]
12         let tail = s[j]
13
14         if (head !== tail) {
15             return false
16         } else {
17             i++
18             j--
19         }
20     }
21     return true
22 };
```

4.1.7 合并两个有序数组

给你两个有序整数数组 `nums1` 和 `nums2`，请你将 `nums2` 合并到 `nums1` 中，使 `nums1` 成为一个有序数组。

说明:

初始化 `nums1` 和 `nums2` 的元素数量分别为 `m` 和 `n`。

你可以假设 `nums1` 有足够的空间（空间大小大于或等于 `m + n`）来保存 `nums2` 中的元素。

示例:

输入:

```
Code block
1 nums1 = [1,2,3,0,0,0], m = 3
2 nums2 = [2,5,6],          n = 3
3
4 输出: [1,2,2,3,5,6]
```

<https://leetcode-cn.com/problems/merge-sorted-array>

4.2 滑动窗口

4.2.1 滑动窗口的最大值

给定一个数组 `nums` 和滑动窗口的大小 `k`，请找出所有滑动窗口里的最大值。

示例:

Code block

```
1 输入: nums = [1,3,-1,-3,5,3,6,7], 和 k = 3
2 输出: [3,3,5,5,6,7]
3 解释:
4
5     滑动窗口的位置                最大值
6     -----
7 [1  3  -1] -3  5  3  6  7         3
8  1 [3  -1  -3] 5  3  6  7         3
9  1  3 [-1  -3  5] 3  6  7         5
10 1  3 -1 [-3  5  3] 6  7         5
11 1  3 -1 -3 [5  3  6] 7         6
12 1  3 -1 -3 5 [3  6  7]         7
```

提示:

你可以假设 `k` 总是有效的，在输入数组不为空的情况下， $1 \leq k \leq$ 输入数组的大小。

链接: <https://leetcode-cn.com/problems/hua-dong-chuang-kou-de-zui-da-zhi-lcof>

滑动窗口，每次左右各滑动一位，并且求窗口中的最大值记录即可，这题坑的地方在于边界情况比较多。

Code block

```
1 let maxSlidingWindow = function (nums, k) {
2   if (k === 0 || !nums.length) {
3     return []
4   }
5   let left = 0
6   let right = k - 1
7   let res = [findMax(nums, left, right)]
8
9   while (right < nums.length - 1) {
10    right++
11    left++
12    res.push(findMax(nums, left, right))
13  }
14
15  return res
16 }
17
18 function findMax(nums, left, right) {
19   let max = -Infinity
20   for (let i = left; i <= right; i++) {
```

```
21     max = Math.max(max, nums[i])
22 }
23 return max
24 }
```

4.2.2 找到字符串中所有字母异位词

给定一个字符串 *s* 和一个非空字符串 *p*，找到 *s* 中所有是 *p* 的字母异位词的子串，返回这些子串的起始索引。

字符串只包含小写英文字母，并且字符串 *s* 和 *p* 的长度都不超过 20100。

说明：

字母异位词指字母相同，但排列不同的字符串。

不考虑答案输出的顺序。

示例 1:

Code block

```
1  输入：
2  s: "cbaebabacd" p: "abc"
3
4  输出：
5  [0, 6]
6
7  解释：
8  起始索引等于 0 的子串是 "cba"，它是 "abc" 的字母异位词。
9  起始索引等于 6 的子串是 "bac"，它是 "abc" 的字母异位词。
```

示例 2:

Code block

```
1  输入：
2  s: "abab" p: "ab"
3
4  输出：
5  [0, 1, 2]
6
7  解释：
8  起始索引等于 0 的子串是 "ab"，它是 "ab" 的字母异位词。
9  起始索引等于 1 的子串是 "ba"，它是 "ab" 的字母异位词。
10 起始索引等于 2 的子串是 "ab"，它是 "ab" 的字母异位词。
```

链接：<https://leetcode-cn.com/problems/find-all-anagrams-in-a-string>

还是典型的滑动窗口去解决的问题，由于字母异位词一定是长度相等的，所以我们需要把窗口的长度始终维持在目标字符的长度，也就是说，每次循环结束后 `left` 和 `right` 是同步前进的。

由于字母异位词不需要考虑顺序，所以只需要运用一个辅助函数 `isAnagrams` 去判断两个 **map** 中记录的字母次数，即可判断出当前位置开始的子串是否和目标字符串形成字母异位词。

Code block

```
1  /**
2   * @param {string} s
3   * @param {string} p
4   * @return {number[]}
5   */
```

```

6  let findAnagrams = function (s, p) {
7    let targetMap = makeCountMap(p)
8    let sl = s.length
9    let pl = p.length
10   // [left,...right] 滑动窗口
11   let left = 0
12   let right = pl - 1
13   let windowMap = makeCountMap(s.substring(left, right + 1))
14   let res = []
15
16   while (left <= sl - pl && right < sl) {
17     if (isAnagrams(windowMap, targetMap)) {
18       res.push(left)
19     }
20     windowMap[s[left]]--
21     right++
22     left++
23     addCountToMap(windowMap, s[right])
24   }
25
26   return res
27 }
28
29 let isAnagrams = function (windowMap, targetMap) {
30   let targetKeys = Object.keys(targetMap)
31   for (let targetKey of targetKeys) {
32     if (
33       !windowMap[targetKey] ||
34       windowMap[targetKey] !== targetMap[targetKey]
35     ) {
36       return false
37     }
38   }
39   return true
40 }
41
42 function addCountToMap(map, str) {
43   if (!map[str]) {
44     map[str] = 1
45   } else {
46     map[str]++
47   }
48 }
49
50 function makeCountMap(strs) {
51   let map = {}
52   for (let i = 0; i < strs.length; i++) {
53     let letter = strs[i]
54     addCountToMap(map, letter)
55   }
56   return map
57 }

```

4.2.3 最小覆盖子串

给你一个字符串 S、一个字符串 T，请在字符串 S 里面找出：包含 T 所有字符的最小子串。

示例：

Code block

```
1  输入：S = "ADOBECODEBANC", T = "ABC"
```


说明:

如果 S 中不存这样的子串, 则返回空字符串 ""。

如果 S 中存在这样的子串, 我们保证它是唯一的答案。

<https://leetcode-cn.com/problems/minimum-window-substring>

根据目标字符串 `t` 生成一个目标 map, 记录每个字符的目标值出现的次数。

然后就是维护一个滑动窗口, 并且针对这个滑动窗口中的字符也生成一个 map 去记录字符出现次数。

每次循环都去对比窗口的 map 里的字符是否能覆盖目标 map 里的字符。

覆盖的意思就是, 目标 map 里的每个字符在窗口 map 中出现, 并且出现的次数要 $\geq$ 目标 map 中此字符出现的次数。

窗口滑动逻辑:

1. 如果当前还没有能覆盖, 那么就右滑右边界。
2. 如果当前已经覆盖了, 记录下当前的子串, 并且右滑左边界看看能否进一步缩小子串的长度。

两种情况下停止循环, 返回结果:

1. 左边界达到 `给定字符长度 - 目标字符串的长度`, 此时不管匹配与否, 都是最短能满足的了。
2. 右边界超出给定字符的长度, 这种情况会出现在右边界已经到头了, 但是还没有能覆盖目标字符串, 此时就算继续滑动也不可能得到结果了。

Code block

```
1  var findAnagrams = function(s, p) {
2    let sL = s.length, pL = p.length
3    if (sL < pL) return [];
4
5    let left = 0, right = 0, // 滑动窗口两侧索引
6        windowMap = new Map(), // 统计滑动窗口内每个 char 的数量
7        pMap = new Map(), // 统计字符串 p 内每个 char 的数量
8        valid = 0, // 滑动窗口 和 字符串 p 中数量相等的 char 的个数
9        ret = [] // 结果集
10
11    for (const char of p) {
12      pMap.set(char, (pMap.get(char) || 0) + 1)
13    }
14
15    while (right < sL) {
16      let char = s[right++]
17      if (pMap.has(char)) {
18        windowMap.set(char, (windowMap.get(char) || 0) + 1);
19        (windowMap.get(char) === pMap.get(char)) && valid++
20      }
21
22      if (right - left === pL) {
23        // 滑动窗口内的字符数和 pL 相等时, 并且 2 个 map 中每个 char 的数量都相等, 说明找到异位词了。
24        valid === pMap.size && ret.push(left)
25
26        // 无论有没有找到异位词, left 都要右移了。
27        let char = s[left++]
28        if (pMap.has(char)) {
29          // 如果 pMap 中有 char, 并且 pMap 中 char 的数量和 windowMap 中的相等, (因为 left 要右移了, 所以)
30          valid--,
31          (windowMap.get(char) === pMap.get(char)) && valid--;
32          // 因为 left 要右移了, 所以 windowMap 中的 char 数量也 - 1,
33          windowMap.set(char, windowMap.get(char) - 1);
34        }
35      }
36      right++
37    }
38    return ret
39  }
```

```
33     }
34     }
35 }
36 return ret
37 };
```

4.2.4 无重复字符的最长子串

给定一个字符串，请你找出其中不含有重复字符的 最长子串 的长度。

示例 1:

Code block

```
1  输入: "abcabcbb"
2  输出: 3
3  解释: 因为无重复字符的最长子串是 "abc", 所以其长度为 3。
```

示例 2:

Code block

```
1  输入: "bbbbbb"
2  输出: 1
3  解释: 因为无重复字符的最长子串是 "b", 所以其长度为 1。
```

示例 3:

Code block

```
1  输入: "pwwkew"
2  输出: 3
3  解释: 因为无重复字符的最长子串是 "wke", 所以其长度为 3。
4      请注意, 你的答案必须是 子串 的长度, "pwke" 是一个子序列, 不是子串。
```

链接: <https://leetcode-cn.com/problems/longest-substring-without-repeating-characters>

这题是比较典型的滑动窗口问题，定义一个左边界 `left` 和一个右边界 `right`，形成一个窗口，并且在这个窗口中保证不出现重复的字符串。

这需要用到一个新的变量 `freqMap`，用来记录窗口中的字母出现的频率数。在此基础上，先尝试取窗口的右边界再右边一个位置的值，也就是 `str[right + 1]`，然后拿这个值去 `freqMap` 中查找：

1. 这个值没有出现过，那就直接把 `right ++`，扩大窗口右边界。
2. 如果这个值出现过，那么把 `left ++`，缩进左边界，并且记得把 `str[left]` 位置的值在 `freqMap` 中减掉。

循环条件是 `left < str.length`，允许左边界一直滑动到字符串的右界。

Code block

```
1  /**
2   * @param {string} s
3   * @return {number}
4   */
5  let lengthOfLongestSubstring = function (str) {
6      let n = str.length
7      // 滑动窗口为s[left...right]
8      let left = 0
```

```
9    let right = -1
10   let freqMap = {} // 记录当前子串中下标对应的出现频率
11   let max = 0 // 找到的满足条件子串的最长长度
12
13   while (left < n) {
14     let nextLetter = str[right + 1]
15     if (!freqMap[nextLetter] && nextLetter !== undefined) {
16       freqMap[nextLetter] = 1
17       right++
18     } else {
19       freqMap[str[left]] = 0
20       left++
21     }
22     max = Math.max(max, right - left + 1)
23   }
24
25   return max
26 }
```

5. 前端项目经验相关面试考察点

1. 所谓的面试，是在一众简历中选择最合适的候选人。面试之所以卷，不是指简历的内容写的多或者项目多就好，而是在跟其他人竞争时，你有别人的简历中所没有的亮点，能够体现出你的技术水平，这才算好。所谓的行情不好，只是原先从简历中选择 top 50%降低到了top 25%，但有亮点的简历，始终是在top 20%以内的，所以没有亮点和深度的简历之间越来越卷，越来越供大于求；但有亮点有技术深度的人始终是处于供不应求的情况；
2. 面试时，技术水平的高低始终是第一位的，如果水平足够，但学历低，优先级也会比学历高但水平低的人高；当然，如果水平一样，肯定有限学历高的候选人；
3. 现在的面试都是通过项目切入，根据项目中具体的实现的功能来引导使用的技术栈，考察的是针对对应的技术栈，具体掌握的深度和广度；
4. 面试的时间一般不会超过1小时，在1小时里不可能把所有的前端内容全部问完，所以面试官会挖掘候选人的项目亮点，通过候选人的面试，考察候选人对知识技能的掌握，再逐步提升难度，摸透候选人对知识掌握的边界；
5. 面试的过程是发掘出候选人优势的过程，不是打压候选人的过程，所以不用担心个别地方说的不好会不会导致面试不过，如果亮点项目很充分，能够体现出个人水平，面试官也不会在意一些边支末节的。

项目经验通常和个人经历关系比较大，前端业务相关的的一些项目经验通常包括管理端、H5 直出、Node.js、可视化，另外还包括参与工具开发的经验，方案选型、架构设计等。

项目相关的内容，比如性能优化、前端框架之类的

5.1 前端框架与工具库

首先我们来看看前端框架，不管你开发管理端、PC Web、H5，还是现在比较流行的小程序，总会面临要使用某一个框架来开发。因此，以下的问题可能与你有关：

- 谈谈你对前端常见的框架（Angular/React/Vue）的理解
- 该项目使用 Angular/React/Vue 的原因是
- 如果现在你重新决策，你会使用什么框架
- 你有了解过这些框架都做了哪些事情，介绍一下是怎么实现的
- Vue 中的双向绑定是怎么实现的？
- 讲讲 React 的虚拟 DOM
- 如何进行状态管理，Vuex/Redux/Mobx 等工具是怎么做的
- 单页应用是什么？路由是如何实现的
- 如何进行 SEO 优化

如果你使用到了小程序，还可能会问到：

- 小程序和 H5 有什么不一样，为什么选小程序而不是 H5
- 有考虑在小程序里嵌 H5 实现吗，为什么
- 为什么小程序的性能要好一些
- 小程序开发有用到哪些框架吗、为什么

而工具库相关的就太多了，一般会这么问：

- 有实际使用过哪些第三方库
- 这些工具库有什么特性和优缺点

项目相关的许多问题，其实是我们工作中经常会遇到并需要进行思考的问题。如果平时有养成思考和总结的习惯，那么这些问题很容易就能回答出来。如果平时工作中比较少进行这样的思考，也可以在面试准备的时候多关注下。

5.2 Node.js 与服务端

Node.js 相关的可能包括：

- 为什么要用 Node.js（而不是 PHP/JAVA/GO/C++等）
- Node.js 有哪些特点，单线程的优势和缺点是什么
- Node.js 有哪些定时功能
- `Process.nextTick` 和 `setImmediate` 的区别
- Node.js 中的异步和同步怎么理解，异步流程如何控制
- 简单介绍一下 Node.js 中的核心内置类库(事件，流，文件，网络等)
- express 是如何从一个中间件执行到下一个中间件的
- express、koa、egg 之间的区别
- Rest API 有使用过吗，介绍一下

以上这些都属于很基础的问题。很多时候，我们会使用 Node.js 去做一些脚本工程或是服务端接入层等工作。如果项目中有使用 Node.js，面试官更多会结合项目相关的进行提问。

5.3 性能优化

性能优化的其实跟项目比较相关，常见的包括：

- 有做过性能优化相关的项目吗，具体的优化过程是怎样的/优化效果是怎样的
- 常见的性能优化包括哪些内容
- 如何理解项目的性能瓶颈/什么时候我们需要对一个项目进行优化
- 图片加载性能有哪些可以优化的地方
- 要怎么做好代码分割/降低代码包大小可以有哪些方式
- 首屏页面加载很慢，要怎么优化
- Tree Shaking 是怎样一个过程
- 页面有没有做什么柔性降级的处理

很多时候，性能优化也是与项目本身紧紧相关，一般来说会包括首屏耗时优化、页面内容渲染耗时优化、内存优化等，可能涉及代码包大小、下载耗时、首屏直出、存储资源（内存/indexDB）等内容。

5.4 前端工程化

如今前端工程化的趋势越来越重，通常从脚手架开始：

- 为什么我们开发的时候要使用脚手架
- 如何理解模块化

- 为什么要使用 Webpack，它和 Gulp 的区别是
- 讲一下 Webpack 中常用的一些配置、Loader、插件
- Babel 的作用是什么，如何选择合适的 Babel 版本
- Webpack 是怎么将多个文件打包成一个，依赖问题如何解决
- 有写过 Webpack 插件吗，Webpack 编译的过程具体是怎样的
- CSS 文件打包过程中，如何避免 CSS 全局污染
- 本地开发和代码打包的流程分别是怎样的

除了脚手架相关，如今自动化、流程化的使用也越来越多了：

- 怎么理解持续集成和持续部署
- 你们的项目有使用 CI/CD 吗，为什么
- 你们的代码有写单元测试/自动化测试吗，为什么
- 前端代码支持自动化发布吗，如何做到的

工程化和自动化是如今前端的一个趋势，由于团队协作越来越多，如何提升团队协作的效率也是一个可具备的技能。

5.5 开发效率提升

效能提升的意识在工作中很重要，大家都不喜欢低效的加班。通常可能问到的问题包括：

- 做了很多的管理端/H5，有考虑过怎么提升开发效率吗
- 你的项目里，有没有哪些工作是可以的工具完成的
- 项目中有进行组件和公共库的封装吗
- 如何管理这些公共组件/工具的兼容问题
- 日常工作中，如何提升自己的工作效率

5.6 监控、灰度与发布

发布和监控这部分，可能较大的业务才会有，涉及的问题可以有：

- 日常开发过程中，怎么保证页面质量
- 版本发布有进行灰度吗？灰度的过程是怎样的
- 版本发布过程中，如何及时地发现问题
- 发生异常，要怎么快速地定位到具体位置
- 如何观察线上代码的运行质量

对于大型项目来说，灰度发布几乎是开发必备，而监控和问题定位也需要各式各样的工具来辅助优化。

5.7 多人协作

一些较大的项目，通常由多个开发合作完成。而多人协作的经验也很有帮助：

- 多人开发过程中，代码冲突如何解决
- 项目中有使用 Git 吗？介绍一下 Git flow 流程
- 如果项目频繁交接，如何提升开发效率
- 有遇到代码习惯差异的问题吗，如何解决
- 有哪些常用的代码校验的工具
- 怎么强制进行 Code Review

看到这么多内容不要慌，一般来说面试官只会根据你的工作经历来询问对应的问题，所以如果你并没有完全掌握某一块的内容，请不要写在简历上，你永远也不知道面试官会延伸到哪

6. 师资&课程内容

- 1. 师资：一线大厂/外企现役P8级高级专家亲自带；
- 2. 内容：深度对标阿里P6-P7，涵盖大厂所有内训标准；

6.1 项目实战内容

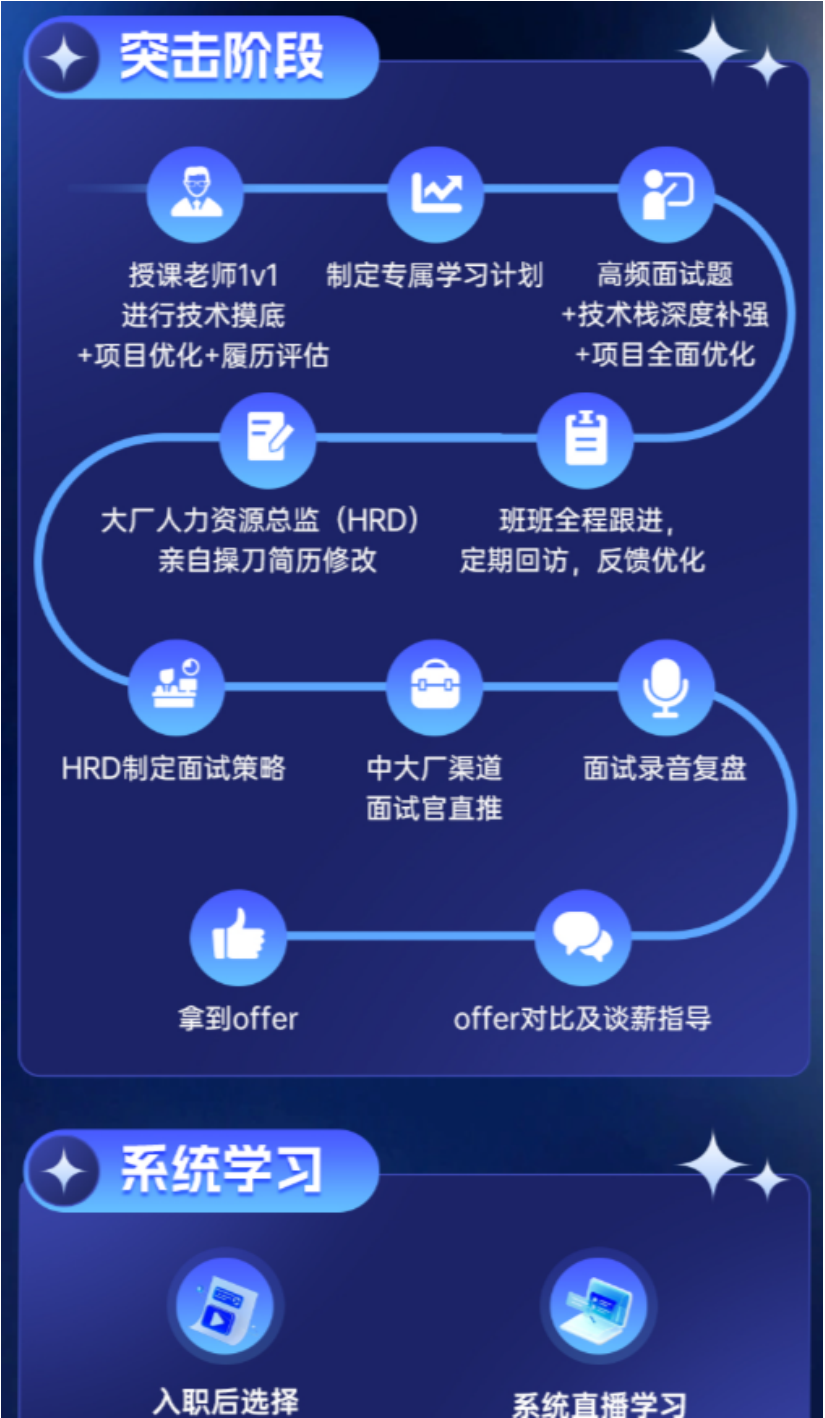
github: <https://github.com/encode-studio-fe>

📖 前端实战课分享汇总 （可咨询班主任要密码权限）

6.2 服务内容

系统&突击两手抓

全程定制，学习计划+简历指导+面试指导+突击指导+全国内推+试用期全程陪跑，全网独家2V1模式，双师辅导（现役P8+HRD）



6.3 专业一对一

近期一对一

- 📖 (2025-02-06) xxx-2.5年-React-24K
- 📖 (2025-02-07) xxx-1年-React-15k
- 📖 (2025-01-05) xxx-7年-Vue-20k

报名学员专享

📖 13. 如何写出高质量的简历 (For 付费学员)

6.4 渠道内推 & 保障就业

渠道：面试官直推/高管直推（简历直达leader）

保障：合同保障最低薪资涨幅，保障心仪企业offer，无限次直推，直到满意为止

6.5 近期报名学员案例

